

Synthesizing a 16-bit Fibonacci Generator

Speed vs. area experiments and the critical path

Bibek Shrestha | September 2026 | Synopsys VCS + Design Compiler, SAED 90 nm

The assignment

Use the SystemVerilog Fibonacci generator from Lab 1. Synthesize it with the flow from the tutorial. Experiment with different optimizations (for speed or for area). Verify that the synthesized design passes the Lab 1 testbench. Generate area and speed reports. **What is the critical path in your design?**

At a glance

	Baseline	Speed	Area
Clock target	5 ns	3 ns	5 ns
Effort (map / area)	medium / medium	high / low	medium / high
Critical path delay	4.35 ns	3.01 ns	4.35 ns
Slack	+0.56 ns MET	-0.09 ns VIOLATED	+0.56 ns MET
Total area	2759.82	3302.43	2750.85
Cells	122	201	122
Adder built as	ripple carry	NAND2/NAND3 chain	ripple carry

Table 1: Results of the three synthesis runs.

Answer: the critical path

current_reg[1] → 16-bit ripple-carry adder r53 (14 full adders + XOR for bit 15) → enable mux A022X1 → OUT_reg[15]. Delay 4.35 ns, slack +0.56 ns at 5 ns, so the design can run at about **225 MHz**.

1 The design

FIBO outputs 1, 1, 2, 3, 5, 8, 13, ... on a 16-bit bus. It has a clock, an asynchronous reset `rst`, and an enable `en`. Each enabled clock does:

```
previous <= current;
current  <= current + previous;
OUT      <= current + previous;
```

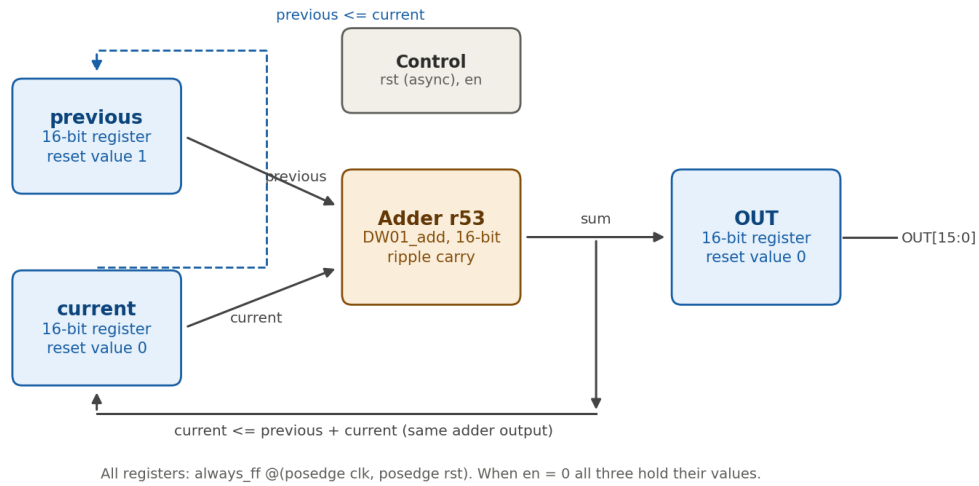


Figure 1: What the RTL turns into: three 16-bit registers (48 flip-flops) and one shared adder.

Cycle	previous	current	sum	OUT
1	1	0	1	1
2	0	1	1	1
3	1	1	2	2
4	1	2	3	3
5	2	3	5	5
6	3	5	8	8
7	5	8	13	13

Table 2: How the registers step through the sequence after reset.

2 Workflow

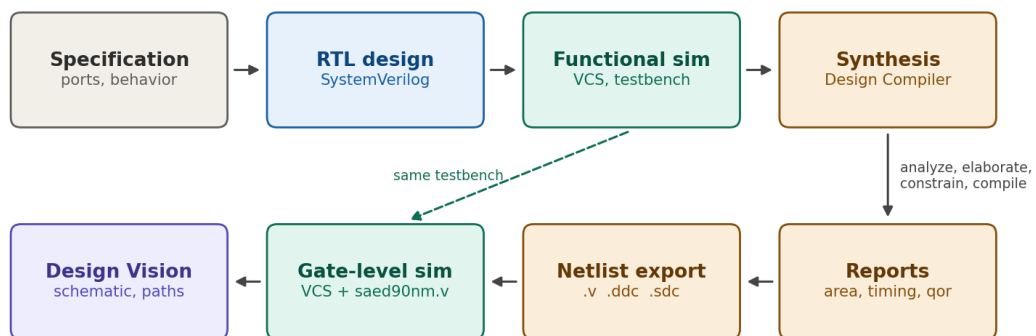


Figure 2: The flow used in all Lab 2 assignments.

1. **Functional simulation** of the RTL with the Lab 1 testbench.
2. **Synthesis** with one script, `fibo_flow.tcl`. Four lines at the top pick the variant, and every report and netlist gets the variant name, so runs never overwrite each other.

3. **Reports:** area, timing, QoR, resources and constraints for each run.
4. **Gate-level simulation** of the netlist with the same testbench.
5. **Design Vision** to look at the synthesized circuit.

```
vcs -sverilog -full64 src/assignment2.sv src/assignment2_tb.sv -o f_sim/fibo.simv
./f_sim/fibo.simv
dc_shell -f fibo_flow.tcl           # once per variant
```

Constraints used: clock period T (5 or 3 ns), input and output delay $0.4T$, set_max_area 0, library saed90nm_typ.

3 RTL simulation

Time (ns)	en	OUT	Note
0, 1	0	0	reset, then idle
3, 7, 9, 11	1	1, 2, 3, 5	counting
13	0	5	held (en = 0)
15, 17	1	8, 13	counting again
19	0	0	final reset

Table 3: The RTL gives the right sequence, including the hold.

4 Synthesis results

Area

	Baseline	Speed	Area
Combinational area	1070.90	1570.41	1065.37
Flip-flop (non-comb.) area	1547.37	1547.37	1547.37
Net area	141.55	184.66	138.11
Total area	2759.82	3302.43	2750.85
Comb. / sequential cells	72 / 48	151 / 48	72 / 48

Table 4: Area reports. The 48 flip-flops cost the same in every run; only the logic changes.

Timing

Slack is simply the time left over before the clock edge:

$$\text{slack} = (T - t_{\text{setup}}) - t_{\text{arrival}} \quad \Rightarrow \quad (5 - 0.09) - 4.35 = +0.56 \text{ ns}$$

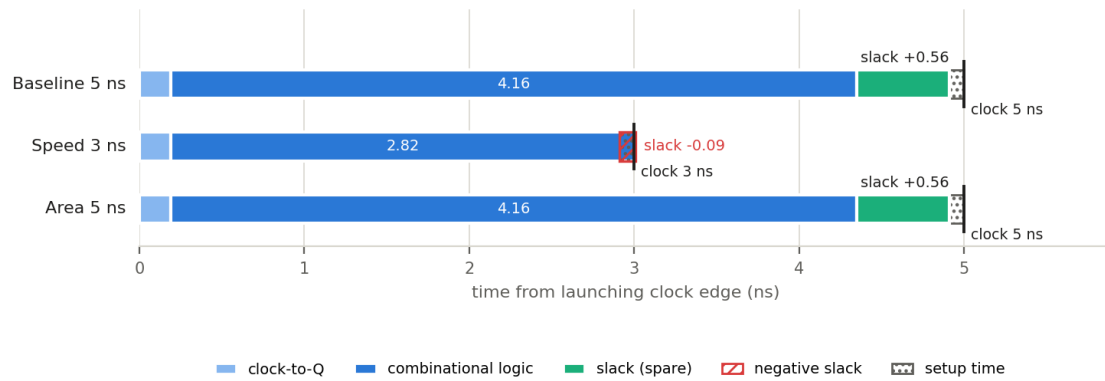


Figure 3: Where the time goes on the critical path in each run.

The report lists the path gate by gate. Each full adder adds about 0.27 ns, and there are 14 of them in a row:

current_reg[1]/Q (DFFARX1)	0.19	0.19
r53/U1_1/CO (FADDX1)	0.39	0.58
r53/U1_2/CO (FADDX1)	0.27	0.85
... 11 more full adders, 0.27 each ...		
r53/U1_14/CO (FADDX1)	0.26	4.07
r53/U1_15/Q (XOR3X1)	0.14	4.21
U27/Q (AO22X1)	0.11	4.32
OUT_reg[15]/D (DFFARX1)	0.03	4.35
slack (MET)		0.56

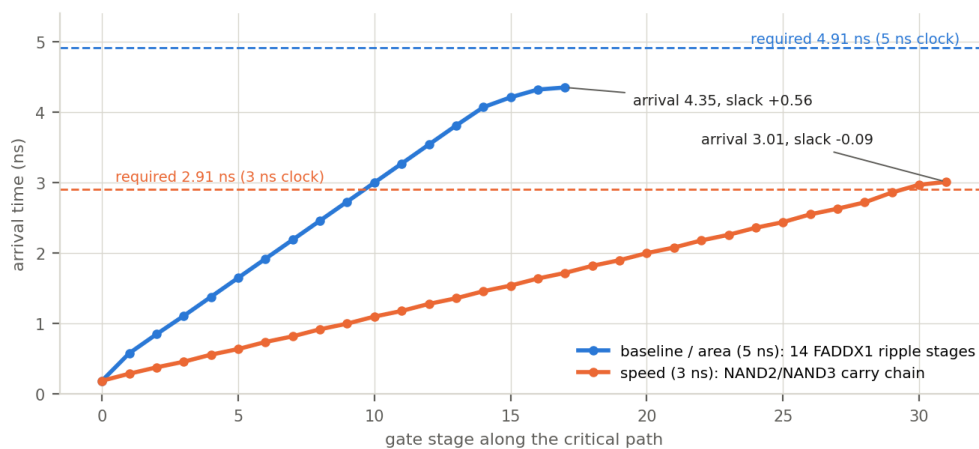


Figure 4: Arrival time along the critical path: ripple adder (blue) vs. the faster chain DC built for 3 ns (orange).

Other things the reports showed

- **One adder, not two.** The resources report maps both current + previous expressions onto one DW01_add (rpl = ripple).
- **Mixed reset cells.** previous resets to 1, so bit 0 uses a set-type flip-flop (DFFASX1) and bits 1 to 15 use clear-type ones (DFFARX1).
- **max_area VIOLATED is normal.** Asking for zero area can never be met; it just means “as small as possible”.
- **check_design:** one harmless LINT-1 warning (an unused cell removed by compile).

5 Speed vs. area

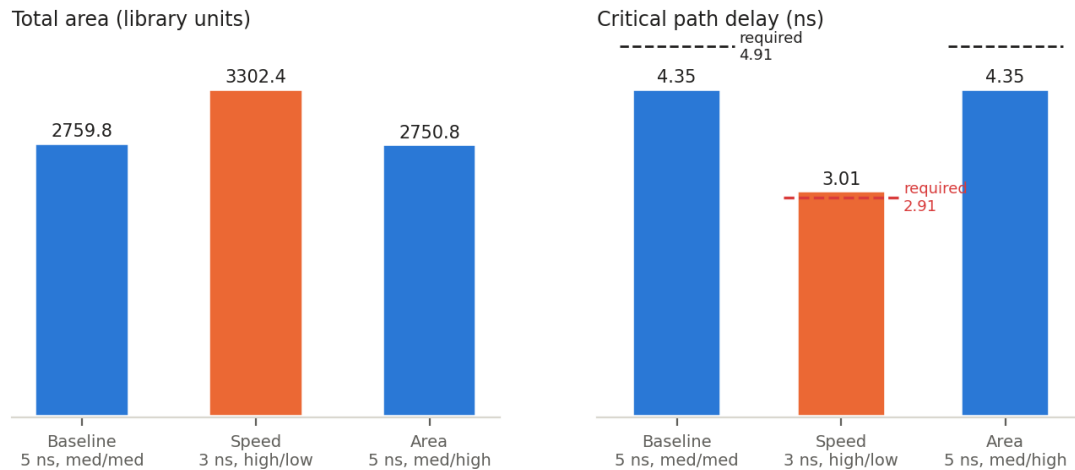


Figure 5: Area and delay of each run. Dashed lines are the required times.

- **Speed run:** to aim for 3 ns, DC replaced the ripple adder with a faster NAND-based carry chain. The path got **31% faster** (4.35 → 3.01 ns) but the design got **20% bigger** and used 79 more cells. It still missed 3 ns by 0.09 ns.
- **Area run:** timing was already met, so there was little to remove: only 9 units saved.

6 Gate-level simulation

```
vcs -sverilog -full64 -debug_access+all src/assignment2_tb.sv output/fibo_area_synth.v \
.../verilog/saed90nm.v -o ps_sim/fibo_synth.vsim
./ps_sim/fibo_synth.vsim
```

Time (ns)	RTL OUT	Gate-level OUT	
3	1	0	one cycle late
7 / 9 / 11	2 / 3 / 5	1 / 2 / 3	one cycle late
17	13	8	one cycle late
19	0	0	13 never appears

Table 5: The netlist gives the right values, but one clock later than the RTL.

Why: the testbench changes `en` and `rst` right after `@(posedge clk)`, on the *same* edge where the flip-flops read them. It is a tie, and the RTL and gate-level flip-flop models break the tie differently. From that edge on the netlist runs one cycle behind, and the final reset arrives before 13 is captured. This is a testbench issue, not a synthesis bug.

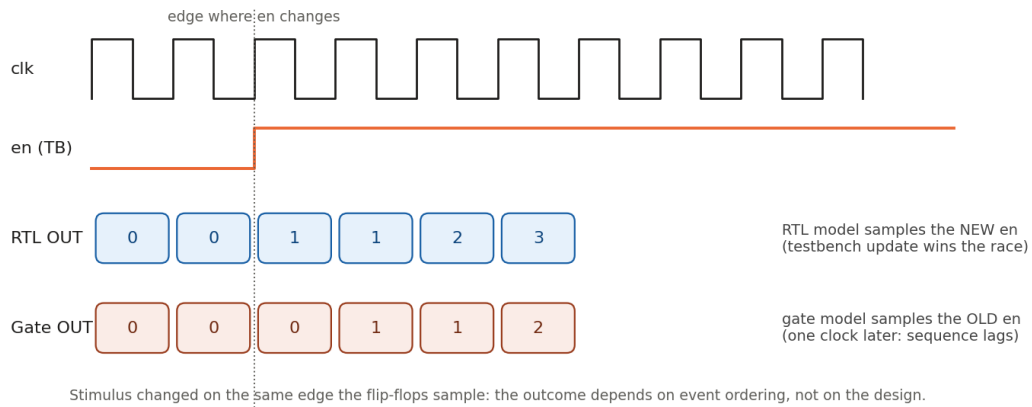


Figure 6: The same input edge is read differently by the two models.

Fix

Change inputs on the falling edge (`@(negedge clk)`), as the Assignment 2 and 3 testbenches do. Their gate-level runs match the RTL exactly. Also, a 5 ns clock in the testbench would match the synthesis target.

7 Design Vision

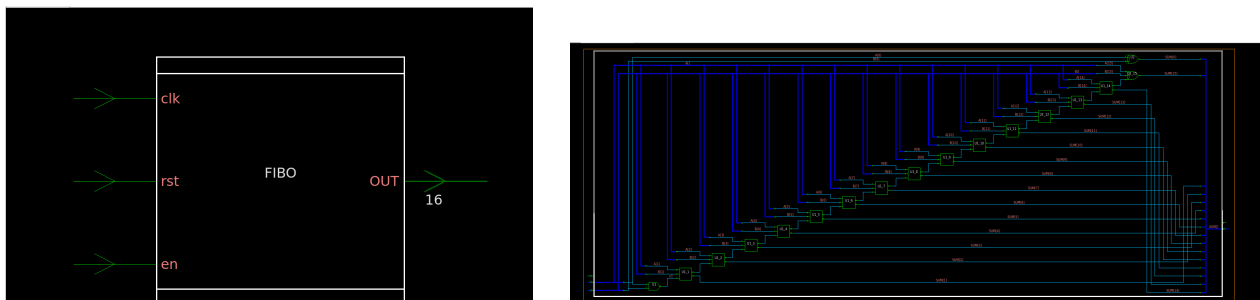


Figure 7: Left: synthesized FIBO block. Right: gates of the shared adder; the diagonal staircase is the carry chain (the critical path).

8 What I learned

- The clock target decides the hardware: the same code gave a ripple adder at 5 ns and a faster adder at 3 ns.
- Speed costs area: 31% faster cost 20% more area.
- Flip-flops are a fixed cost; synthesis can only optimize the logic.
- A clean timing report does not guarantee a clean simulation. The testbench must not change inputs on the sampling edge.

A Source files

RTL: `src/assignment2.sv`

```

`timescale 1ns/10ps

module FIB0 ( input  logic      clk,
              input  logic      rst,
              input  logic      en,
              output logic [15:0] OUT );

    logic [15:0] previous;
    logic [15:0] current;

    always_ff @(posedge rst, posedge clk)
    begin
        if (rst) begin
            OUT      <= 16'b0;
            current  <= 16'b0;
            previous <= 16'b1;
        end
        else if (en) begin
            previous <= current;
            current  <= current + previous;
            OUT      <= current + previous;
        end
    end
end
endmodule

```

Testbench: src/assignment2_tb.sv

```

// Testbench for fibonacci generator (module assignment2.sv)
`timescale 1ns/10ps

module assignment2_tb;

    logic      clk;
    logic      rst;
    logic      en;
    logic [15:0] OUT;

    FIB0 dut ( .clk(clk), .rst(rst), .en(en), .OUT(OUT) );

    initial clk = 0;
    always #1 clk = ~clk;

    initial begin
        $monitor("Time=%0t | rst=%b en=%b | OUT=%0d ", $time, rst, en, OUT);
        rst = 1;
        en  = 0;
        @(posedge clk);
        rst = 0;
        en  = 0;
        @(posedge clk);
        en  = 1;
        repeat(5) @(posedge clk); // f(0) through f(4)
        en  = 0;
        @(posedge clk); // hold steady
        en  = 1;
        repeat(2) @(posedge clk); // f(5), f(6)
        rst = 1;
        en  = 0;
        @(posedge clk);
        $display("Test finished.");
        $finish;
    end
endmodule

```

```

end

endmodule

```

Synthesis script: src/fibo_flow.tcl

```

# fibo_flow.tcl : Lab 2, Assignment 1 (FIBO)
# Variants: baseline 5/medium/medium, speed 3/high/low, area 5/medium/high
set VARIANT      "area"
set CLK_PERIOD   5
set MAP_EFFORT    medium
set AREA_EFFORT   high

set link_library [list \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_max.db \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_typ.db \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_min.db]
set target_library [list \
  /apps/designlib/SAED90_EDK/SAED_EDK90nm/Digital_Standard_cell_Library/synopsys/models/
  saed90nm_typ.db]

define_design_lib WORK -path work
analyze -library WORK -format sverilog {src/assignment2.sv}
elaborate FIBO -architecture verilog -library WORK
current_design FIBO
link
check_design > reports/fibo_${VARIANT}_check_design.rpt

create_clock clk -name ideal_clock1 -period $CLK_PERIOD
set_input_delay [expr {$CLK_PERIOD * 0.4}] -clock ideal_clock1 \
  [remove_from_collection [all_inputs] clk]
set_output_delay [expr {$CLK_PERIOD * 0.4}] [all_outputs]
set_max_area 0

compile -map_effort $MAP_EFFORT -area_effort $AREA_EFFORT

report_area      > reports/fibo_${VARIANT}_area.rpt
report_timing    > reports/fibo_${VARIANT}_timing.rpt
report_qor       > reports/fibo_${VARIANT}_qor.rpt
report_resources > reports/fibo_${VARIANT}_resources.rpt
report_constraints > reports/fibo_${VARIANT}_constraints.rpt

write -format verilog -hierarchy -output output/fibo_${VARIANT}_synth.v
write_sdc constraints/fibo_${VARIANT}.sdc

report_timing
gui_start
# quit

```